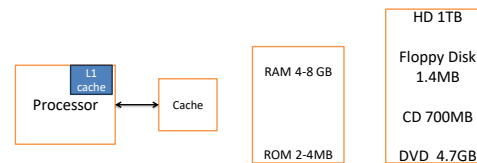


EC340: Computer Organization and Architecture

Memory Hierarchy

1

Memory Hierarchy



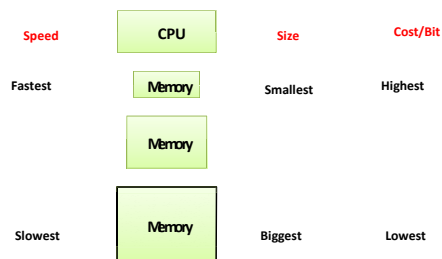
2

Memory technologies

- Semiconductor
 - Registers
 - SRAM
 - DRAM
 - FLASH
 - Magnetic
 - FDD
 - HDD
 - Optical
 - CD
 - DVD
- Random Access (Semiconductor)
- Random + sequential (Magnetic and Optical)

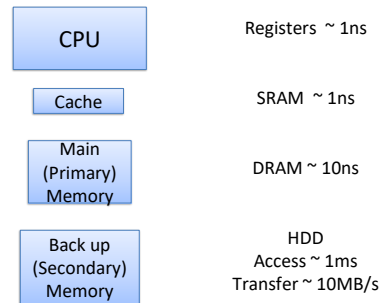
3

Hierarchical structure



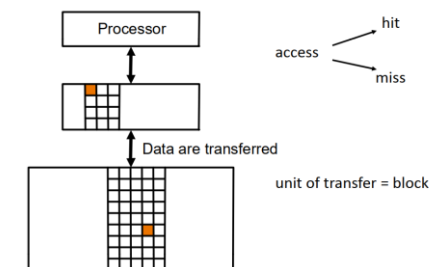
4

Memory Hierarchy



5

Data transfer between levels



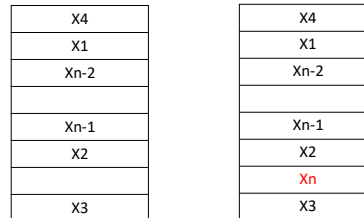
6

Principle of locality

- Temporal Locality
 - references repeated in time
- Spatial Locality
 - references repeated in space
 - Special case: Sequential Locality

7

Cache access example

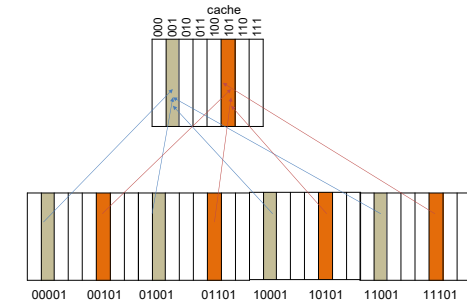


a. Before the reference to Xn

b. After the reference to Xn

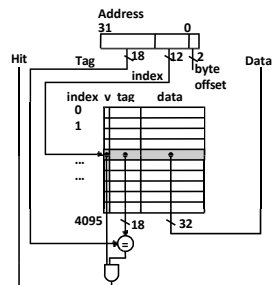
8

Directed mapped cache



9

Cache access mechanism



10

Cache access: Example

Index	V	Tag	Data
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	N		
111	N		

a. The initial state of the cache after power-on

Index	V	Tag	Data
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	Y	10 _{tag}	Memory(10110 _{mem})
111	N		

b. After handling a miss of address (10110_{mem})

Decimal address of reference	Binary address of reference	Hit or miss in cache	Assigned cache block (where found or placed)
22	10110 _{mem}	miss	(10110 _{mem} mod 8) = 111 _{mem}
26	11010 _{mem}	miss	(11010 _{mem} mod 8) = 010 _{mem}
22	10110 _{mem}	hit	(10110 _{mem} mod 8) = 111 _{mem}
26	11010 _{mem}	hit	(11010 _{mem} mod 8) = 010 _{mem}
16	10000 _{mem}	miss	(10000 _{mem} mod 8) = 000 _{mem}
3	00011 _{mem}	miss	(00011 _{mem} mod 8) = 011 _{mem}
16	10000 _{mem}	hit	(10000 _{mem} mod 8) = 000 _{mem}
18	10010 _{mem}	miss	(10010 _{mem} mod 8) = 010 _{mem}

11

Cache access: Example

Index	V	Tag	Data
000	N		
001	N		
010	Y	11 _{tag}	Memory(11010 _{mem})
011	N		
100	N		
101	N		
110	Y	10 _{tag}	Memory(10110 _{mem})
111	N		

c. After handling a miss of address (11010_{mem})

Index	V	Tag	Data
000	Y	10 _{tag}	Memory(10000 _{mem})
001	N		
010	Y	11 _{tag}	Memory(11010 _{mem})
011	Y	00 _{tag}	Memory(00011 _{mem})
100	N		
101	N		
110	Y	10 _{tag}	Memory(10110 _{mem})
111	N		

e. After handling a miss of address (00011_{mem})

Index	V	Tag	Data
000	Y	10 _{tag}	Memory(10000 _{mem})
001	N		
010	Y	10 _{tag}	Memory(101010 _{mem})
011	Y	00 _{tag}	Memory(00011 _{mem})
100	N		
101	N		
110	Y	10 _{tag}	Memory(10110 _{mem})
111	N		

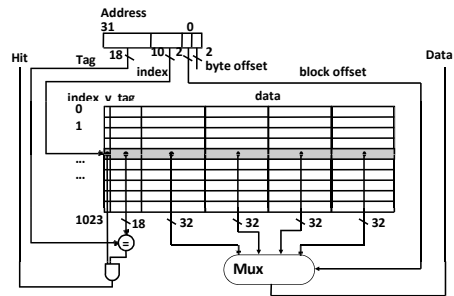
d. After handling a miss of address (10000_{mem})

Index	V	Tag	Data
000	Y	10 _{tag}	Memory(10000 _{mem})
001	N		
010	Y	10 _{tag}	Memory(101010 _{mem})
011	Y	00 _{tag}	Memory(00011 _{mem})
100	N		
101	N		
110	Y	10 _{tag}	Memory(10110 _{mem})
111	N		

f. After handling a miss of address (10010_{mem})

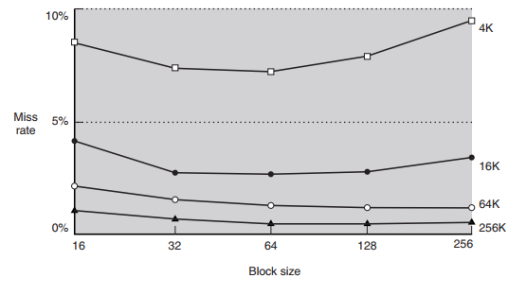
12

Cache with 4 word blocks



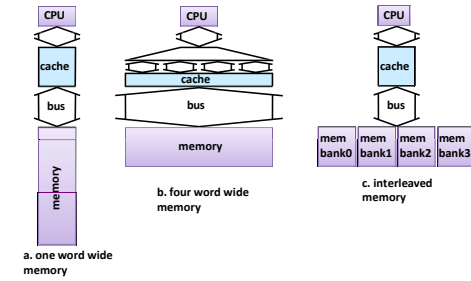
13

Miss rate vs block size



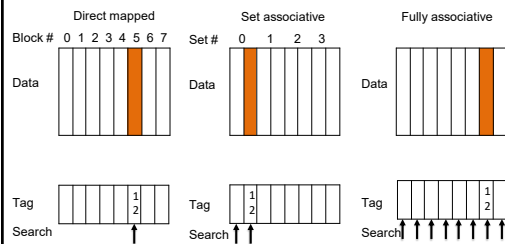
14

Transferring blocks to/from memory



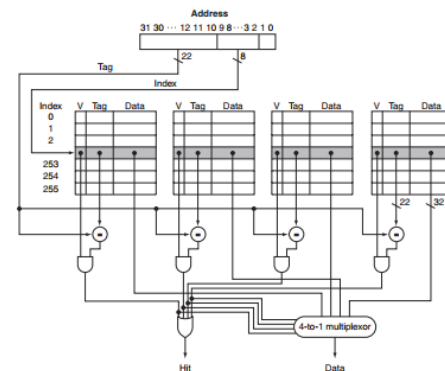
15

More flexible block placement



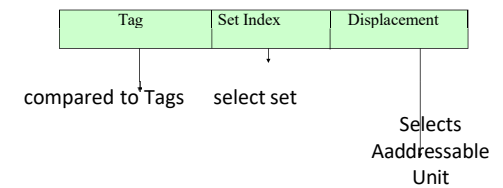
16

4-way set associate cache



17

Addressing Cache



18

Memory Hierarchy Analysis

Memory M_i : $M_1, M_2, \dots, M_n$
 Capacity S_i : $S_1 < S_2 < \dots < S_n$
 Unit cost C_i : $C_1 > C_2 > \dots > C_n$

Access time t_i : $\tau_1 + \tau_2 + \dots + \tau_i$ (τ_i at level i)
 $\tau_1 < \tau_2 < \dots < \tau_n$

Hit ratios $h_i(s_i)$: $h_1 < h_2 < \dots < h_n = 1$
 Effective time T_{eff} : $\sum_i m_i \cdot h_i \cdot t_i = \sum_i m_i \cdot \tau_i$
 Miss before level i, m_i : $(1-h_1)(1-h_2) \dots (1-h_{i-1})$

19

Two level case

M1 is cache, M2 is main memory

Access time t_1 : τ_1
 Access time t_2 : $\tau_1 + \tau_2$
 Hit ratios: $h_1 < h_2 = 1$
 Miss before level 1, m_1 : 1
 Miss before level 2, m_2 : $1-h_1$
 Effective time T_{eff} : $m_1 \cdot h_1 \cdot t_1 + m_2 \cdot h_2 \cdot t_2 =$
 $h_1 \cdot t_1 + (1-h_1) \cdot t_2 = \tau_1 + (1-h_1) \cdot \tau_2$

20

Performance

$T_{eff} = \tau_1 + (1-h_1) \cdot \tau_2$
 Average memory access time =
 Hit time + Miss rate * Miss penalty

Program execution time =
 $\text{InstrCount} \cdot \text{Cycletime} \cdot (\text{CPI}_{\text{exec}} + \text{Memstalls}/\text{Instr})$

Mem stalls/Instr =
 Miss rate * Miss Penalty * Mem accesses/Instr

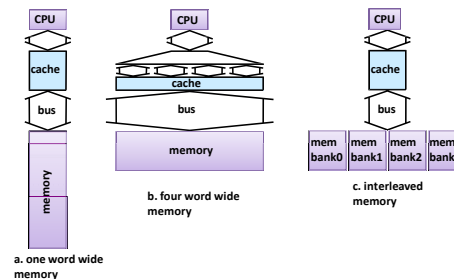
21

Performance Improvement

- Reducing miss penalty
- Reducing miss rate
- Reducing miss penalty * miss rate
- Reducing hit time

22

Transferring blocks to/from memory



23

Miss penalty example

- 1 clock cycle to send address
- 15 cycles for RAM access
- 1 cycle for sending data
- Block size = 4 words

Miss penalty:

Case(a): $4(1+15+1) = 68$

Case(b): $1+1(15+1) = 17$

Case(c): $1+ 15+4*1 = 20$

If Memory and bus were 2
word wide
Case(b): $2(1+15+1) = 34$

If degree if interleaving is 2
then
Case(c): $2(1+15+2) = 36$

24

Cache policies

- Read
 - Sequential/ Concurrent, Simple/Forward
- Load
 - Block load/ Load Forward/Wrap around
- Fetch
 - Demand fetch, hw prefetch, sw prefetch
- Replacement
 - LRU/LFU/FIFO/Random
- Write
 - Write back/Write through

25

Read policies

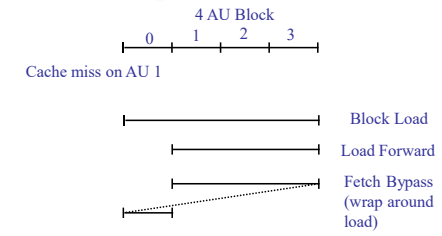
How cache handles read request for data that is not currently in the cache.

- Sequential or concurrent
 - processor stalls and waits for the required data to be fetched from main memory.
 - The processor fetches the required data while simultaneously continuing to execute instructions that do not depend on that missing data
- With or without forwarding
 - cache fetches the exact block of data that was requested.
 - cache specifically requests the exact word the processor needs first. This "critical word" is sent to the processor as soon as it arrives from memory, even while the rest of the cache block is still being fetched.

26

Load policies

what data is brought into the cache after a read miss.



27

Fetch Policies

when data is brought into the cache.

- Demand fetching
 - fetch only when required (miss)
- Hardware prefetching
 - automatically prefetch next block
- Software prefetching
 - programmer decides to prefetch

28

Replacement Policies: Only for Associative cache

- Least Recently Used (LRU)
 - Add time stamp to each access
- Least Frequently Used (LFU)
 - Newly added line have frequency 0
 - Add Frequency Ctr for each line access
- First In First Out (FIFO)
 - Choose the in order
- Random
 - Randomly choose

29

Write Policies

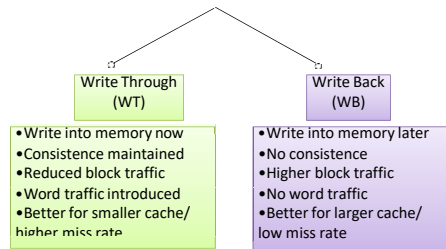
How cache handles **write operations** from the processor.

operations from the processor

- Write Hit
 - Write Back
 - Write Through
- Write Miss
 - Write Back
 - With Write Allocate
 - With No Write Allocate
 - Write Through

30

Write Policies



31

Cache Types

Instruction | Data | Unified | Split

Split vs. Unified:

- Split allows specializing each part
- Unified allows best use of the capacity

On-chip | Off-chip

- on-chip : fast but small (L1/L2 cache)
- off-chip : large but slow (L3 cache)

Single level | Multi level

32

Direct mapped cache demo (block size =1)

Access	16
Hits	3
misses	13

Address references

1	4	8	5	20	17	19	56	9	11	4	43	5	6	9	17
---	---	---	---	----	----	----	----	---	----	---	----	---	---	---	----

---	---	---	---	---	---	---	---	---	---	---	---	H	---	H	H
Hit/miss															

33

Direct mapped cache demo (block size =4)

Access	16
Hits	6
misses	10

Address references

1	4	8	5	20	17	19	56	9	11	4	43	5	6	9	17
---	---	---	---	----	----	----	----	---	----	---	----	---	---	---	----

---	---	---	H	---	---	H	---	---	H	---	---	H	H	-	H
Hit/miss															

34

2-way set assoc cache demo (block size =1)

Access	16
Hits	4
misses	12

Address references

1	4	8	5	20	17	19	56	9	11	4	43	5	6	9	17
---	---	---	---	----	----	----	----	---	----	---	----	---	---	---	----

---	---	---	---	---	---	---	---	---	---	H	---	H	---	H	H
Hit/miss															

35

4-way set assoc cache demo (block size =1)

Access	16
Hits	4
misses	12

Address references

1	4	8	5	20	17	19	56	9	11	4	43	5	6	9	17
---	---	---	---	----	----	----	----	---	----	---	----	---	---	---	----

---	---	---	---	---	---	---	---	---	---	H	---	H	---	H	H
Hit/miss															

36

Types of missess

- Compulsory miss
- Capacity miss
- Conflict miss

37

Sizes and bits

Address size = k bits cache size = S bytes
 Block size = B bytes $B=2^b$
 Degree of S.A. = A
 No. of sets = $(S \div A \cdot B)$
 Index bits = $\log_2(S \div A \cdot B)$
 Bits to address a byte in a block = $\log_2(B)$
 Tag size = $k - \log_2(S \div A \cdot B) - \log_2(B) = k - \log_2(S \div A)$
 Total tag bits = $(k - \log_2(S \div A)) * (S \div B)$

38

Performance analysis example

$$CPI_{eff} = CPI + Miss\ rate * Miss\ penalty * Mem\ accesses / Instr$$

CPI = 1.2 Miss rate = 0.5%
 Block size = 16 w

Miss penalty??
 Memaccess/instr = 1 (assumption)

39

Miss penalty calculation

Data/address transfer time = 1 cycle
 Memory latency = 10 cycles

- a) Miss penalty = $16 * (1 + 10 + 1) = 192$
 b) Miss penalty = $4 * (1 + 10 + 1) = 48$ (if bus size is 4)
 c) Miss penalty = $4 * (1 + 10 + 4 * 1) = 60$

40

CPI calculation

$$CPI_{eff} = 1.2 + .005 * miss\ penalty * 1.0$$

- a) $1.2 + .005 * 192 * 1.0 = 1.2 + .96 = 2.16$
 b) $1.2 + .005 * 48 * 1.0 = 1.2 + .24 = 1.44$
 c) $1.2 + .005 * 60 * 1.0 = 1.2 + .30 = 1.50$

41

Cache comparison example

Cache	mapping	block size	I-miss	D-miss	CPI
1	Direct	1 word	4%	8%	2.0
2	Direct	4 words	2%	5%	??
3	2-way s.a	4 words	2%	4%	??

Miss penalty = 6 + block size (Assumption)

50% instruction have a data reference

Stall cycles: cache1: $7 * (.04 + .08 * 0.5) = 0.56$

cache2: $10 * (.02 + 0.05 * 0.5) = 0.45$

cache3: $10 * (.02 + 0.04 * 0.5) = 0.40$

42

Example continued

Cache	CPI	clockperiod	time/instr
1	2.0	2.0	4.0
2	$2.0 - 0.56 + 0.45 = 1.89$	2.0	3.78
3	$2.0 - 0.56 + 0.40 = 1.84$	2.4	4.416

43

Multi-level cache example

CPI with no miss = 1.0 clock = 500MHz
 Main mem access time = 200ns
 Miss rate = 5%
 Adding L2 cache (20ns) reduces miss to 2%.
 Find performance improvement

Miss penalty (mem) = $200/2 = 100$ cycles
 Effective CPI with L1 = $1 + 5\% * 100 = 6$

44

Example continued

Miss penalty (L2) = $20/2 = 10$ cycles
 Total CPI = Base CPI + stalls due to L1 miss
 + stalls due to L2 miss

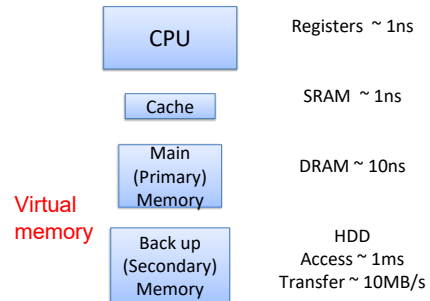
$$= 1.0 + 5\% * 10 + 2\% * 100$$

$$= 1.0 + 0.5 + 2.0 = 3.5$$

$$\text{Performance ratio} = 6.0/3.5 = 1.7$$

45

Memory Hierarchy



46

Virtual memory objectives

- To overcome the **limitation of the physical memory size**, with least inconvenience to the programmer
 - programmer does not bother about moving code & data between disk and main memory, as opposed to "overlays"
- To allow **multiple programs to share memory without interference**

47

What is Virtual Memory?

- An illusion of a memory much larger than the physical (main) memory
 - Each program gets its own virtual address which is larger than physical address space
 - Both spaces divided into equal sized pages - pages in VM and frames in Main memory
 - There may be multiple virtual spaces, one for each program

48

What is Virtual Memory?

- Implemented using hard disk drive
 - Each virtual page has a place on the disk
 - Some of these may also be in the main memory
 - The subset present in the main memory changes from time to time as per the need
- ISA must support large address spaces

49

Virtual memory vs cache : similar but important differences

- **Speed difference**
 - cache and main memory : one order of magnitude
 - main memory and HDD : several orders of magnitude
- **Response to a miss**
 - cache : must be handled by h/w, can't afford to switch context
 - VM : can't keep CPU waiting, must switch context, can be conveniently handled by s/w
- **Terminology differences**
 - page vs block, page fault vs miss, page table vs cache directory etc

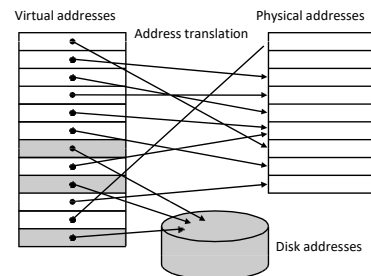
50

Differences contd.

- **Miss rate**
 - VM can only afford extremely low miss rates
 - Main memory much larger than cache, implies much lower miss rate
- **Policies**
 - Pages much larger than blocks (~4KB - 16 KB) : capture large spatial locality, amortize transfer time
 - Fully flexible mapping (not with assoc mem)
 - Always write back
 - Good approximation to LRU

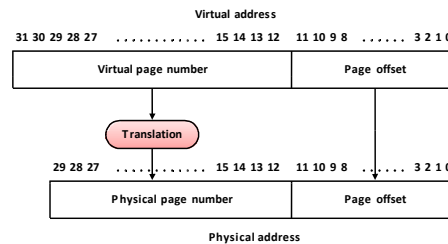
51

Virtual to physical mapping



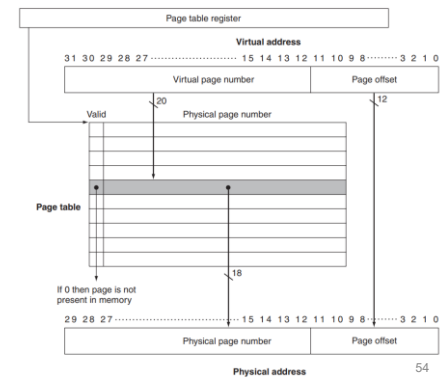
52

Address translation



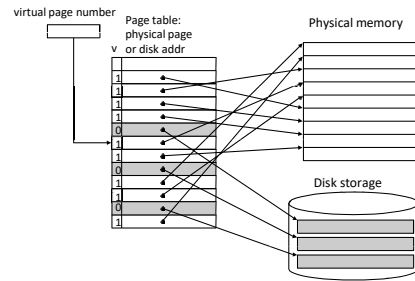
53

Page table



54

Mapping with page table



55

Where is page table stored?

How big is it?

- Suppose virtual address = 32 bits, page size = 4 KB, page table entry = 4 B
- then number of page table entries = number of pages = $2^{32} / 2^{12} = 2^{20}$
- page table size = $2^{20} \times 2^2 = 4 \text{ MB}$
- hundreds / thousands of processes

Store in dedicated memory? main memory?

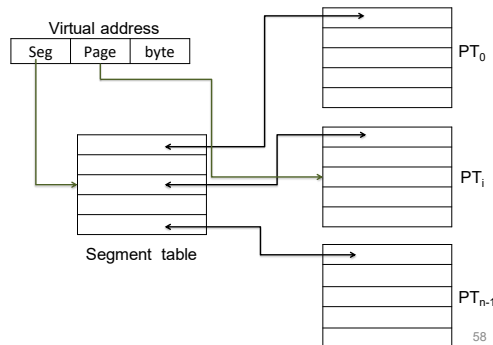
56

Two level page tables

- Virtual space divided into segments
- A smaller page table for each segment
- All page tables need not be in physical memory
- A segment table keeps track of where the page tables of different segments are located

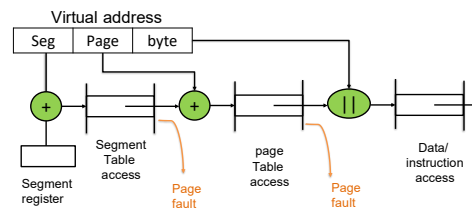
57

2 level page table structure



58

Memory access with 2 level page table



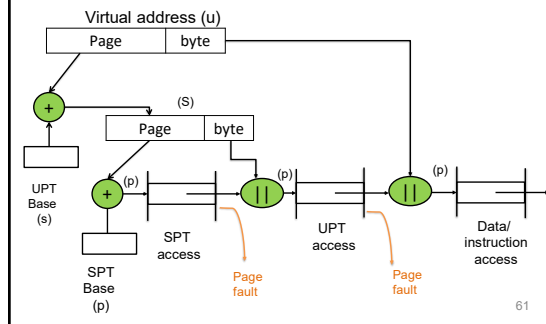
59

Keep table in virtual space

- Page table is paged!
- Only active pages of the page table are brought into physical memory
- How do you locate these pages? - need another page table!
- User page tables are kept in the system's virtual space
- System's page table (or at least a part of it) must be kept in physical memory and accessed directly

60

Memory access with paged page table

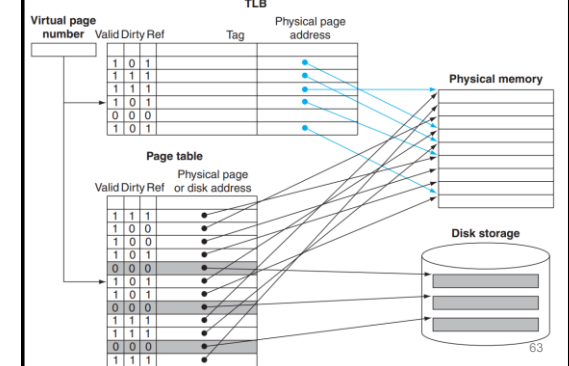


Virtual memory hit time

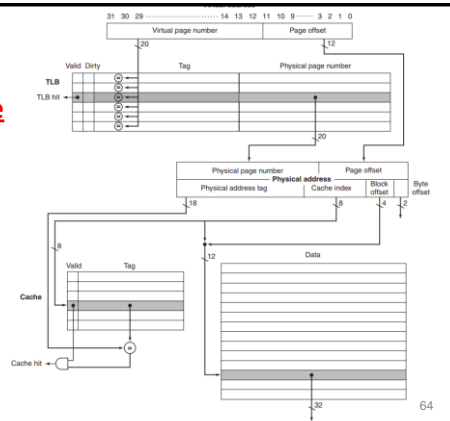
- Simple page table
 - 2x physical memory access time
- 2 level page table
 - 3x physical memory access time
- Paged page table
 - 3x physical memory access time

62

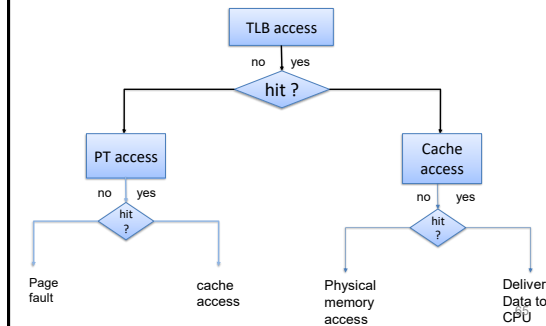
Translation Lookaside Buffer



TLB with Cache



Overall operation



Memory protection with VM

- A process can access only those physical pages which are pointed to by its page table
- Page tables are initialized by the OS and are updated automatically. User processes cant modify these

66